МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: «Технологии защиты информации»

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Разработка примитивной системы защиты на уровне СУБД»

Выполнил студент(ы):

группа.поток ИББД 1.7

Пахомов Владимир Юрьевич / 

ФИО *Подпись*

Сивков Иван Александрович / 

ФИО *Подпись*

Булин Михаил Михайлович / 

ФИО *Подпись*

Проверил:

Ярцева Наталия Андреевна / _____

ФИО *Подпись*

_____

Отметка о выполнении (один из вариантов:

отлично, хорошо, удовлетворительно, зачтено)

_____

Дата

Санкт-Петербург

2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Задачи по мониторингу операций в БД **4**

1.1 Таблица-лог и триггерная функция 4

1.2 Создание триггеров для основных таблиц 4

1.3 Демонстрация работы системы логирования 5

2 Задачи по шифрованию данных **6**

2.1 Таблица с секретными данными 6

2.2 Формирование ключа и шифрование содержимого 6

2.3 Демонстрация невозможности чтения без пароля 6

3 Задачи по разграничению доступа в БД **8**

3.1 Групповые роли для классов потребителей 8

3.2 Назначение привилегий групповым ролям 8

3.3 Индивидуальные роли пользователей 8

3.4 Проверка работы системы разграничения доступа 9

ЗАКЛЮЧЕНИЕ **10**

ВВЕДЕНИЕ

Цель работы: получение навыков создания примитивных систем мониторинга, разграничения доступа и шифрования средствами СУБД.

Объектом защиты выступает база данных `container_cargo_db` информационной системы морских контейнерных перевозок, спроектированная в лабораторной работе №1. База данных включает 9 отношений (*clients*, *ports*, *ships*, *containers*, *orders*, *voyages*, *invoices*, *order_containers*, *voyage_orders*) и 4 представления для двух классов потребителей информации: «Клиент / Грузоотправитель» и «Диспетчер флота / Логист терминала». Реализация выполняется в СУБД PostgreSQL с использованием расширения *pgcrypto*.

В работе решаются три задачи: мониторинг изменений данных с помощью триггеров и таблицы-лога, шифрование таблицы с секретными данными симметричным алгоритмом AES-256 и разграничение доступа на основе ролей по принципу наименьших привилегий.

1 ЗАДАЧИ ПО МОНИТОРИНГУ ОПЕРАЦИЙ В БД

1.1 Таблица-лог и триггерная функция

Создадим таблицу-лог `main_log`, отдельную от основных сущностей базы данных. Для каждой зарегистрированной операции в ней сохраняются тип операции, имя таблицы, время выполнения, роль, от лица которой поступил запрос, а также старые и новые значения изменяемого кортежа в формате JSON. Атрибут `log_item_id` является автогенерируемым идентификатором записи.

```
CREATE TABLE IF NOT EXISTS public.main_log (
    log_item_id    integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, -- автоинкремент
    operation_type text        NOT NULL,                                 -- I / U / D
    table_name     text        NOT NULL,                                 -- в какой таблице
    изменение
    operation_date timestamptz NOT NULL DEFAULT now(),                   -- когда
    user_operator  text        NOT NULL,                                 -- активная роль (
    current_user)
    session_login  text        NOT NULL,                                 -- роль входа в сессию (
    session_user)
    old_data       jsonb,                                                -- старые значения (U, D)
    new_data       jsonb                                                 -- новые значения (I, U)
);
```

Создадим триггерную функцию `logging()`, которая срабатывает при изменениях данных и заносит в таблицу-лог информацию об операции. Для вставки фиксируются только новые значения, для удаления — только старые, для изменения — и старые, и новые. Роль, выполнившая запрос, определяется функциями `current_user` (активная роль) и `session_user` (роль, под которой выполнен вход в сессию).

```
CREATE OR REPLACE FUNCTION public.logging() RETURNS trigger AS $logging$
BEGIN
    IF (TG_OP = 'INSERT') THEN
        INSERT INTO public.main_log (operation_type, table_name, user_operator, session_login,
    old_data, new_data)
        VALUES ('I', TG_TABLE_NAME, current_user, session_user, NULL, to_jsonb(NEW)); -- вставка:
    только новые
    ELSIF (TG_OP = 'UPDATE') THEN
        INSERT INTO public.main_log (operation_type, table_name, user_operator, session_login,
    old_data, new_data)
        VALUES ('U', TG_TABLE_NAME, current_user, session_user, to_jsonb(OLD), to_jsonb(NEW)); --
    изменение: старые и новые
    ELSIF (TG_OP = 'DELETE') THEN
        INSERT INTO public.main_log (operation_type, table_name, user_operator, session_login,
    old_data, new_data)
        VALUES ('D', TG_TABLE_NAME, current_user, session_user, to_jsonb(OLD), NULL); -- удаление:
    только старые
    END IF;
    RETURN NULL; -- для AFTERтриггера- возвращаемое значение не важно
END;
$logging$ LANGUAGE plpgsql;
```

1.2 Создание триггеров для основных таблиц

Для каждой из девяти основных таблиц создадим триггер `logging_exec`, который вызывает функцию `logging()` после любой вставки, изменения или удаления кортежа. Триггер объявлен как построчный (`FOR EACH ROW`), поэтому каждое изменённое кортеж-значение фиксируется отдельной записью в логе. Чтобы не дублировать однотипные команды, триггеры создаются в цикле:

```
DO $$
DECLARE t text;
BEGIN
```

```
    FOREACH t IN ARRAY ARRAY['clients', 'ports', 'ships', 'containers', 'orders',
                             'voyages', 'invoices', 'order_containers', 'voyage_orders'] LOOP
        EXECUTE format(
            'CREATE OR REPLACE TRIGGER logging_exec AFTER INSERT OR UPDATE OR DELETE ON public.%I
    FOR EACH ROW EXECUTE FUNCTION public.logging()',
            t);
    END LOOP;
END $$;
```

Для одной таблицы эквивалентная команда имеет вид:

```
CREATE TRIGGER logging_exec
AFTER INSERT OR UPDATE OR DELETE ON clients
FOR EACH ROW EXECUTE FUNCTION logging();
```

1.3 Демонстрация работы системы логирования

Продемонстрируем работу системы для различных операций и отношений. Для наглядности выполним операции над двумя таблицами — *ports* и *clients*: вставим по одному кортежу, изменим их и удалим. Работа выполняется над собственными тестовыми строками, чтобы не нарушать ограничения внешних ключей.

```
INSERT INTO public.ports (name, country, dock_count, max_draft)
VALUES ('Test Port', 'Russia', 5, 12.5);                                    -- INSERT

INSERT INTO public.clients (name, email, phone, address)
VALUES ('Test Cargo LLC', 'old@test.ru', '+70000000000', 'Saint Petersburg');  -- INSERT

UPDATE public.clients SET email = 'new@test.ru' WHERE name = 'Test Cargo LLC'; -- UPDATE

UPDATE public.ports SET dock_count = 8 WHERE name = 'Test Port';            -- UPDATE

DELETE FROM public.clients WHERE name = 'Test Cargo LLC';                   -- DELETE
DELETE FROM public.ports WHERE name = 'Test Port';                          -- DELETE

SELECT * FROM public.main_log ORDER BY log_item_id;
```

Содержимое таблицы-лога после выполнения операций приведено на рисунке 1.



Рисунок 1 — Логирование операций вставки, изменения и удаления

В таблице зафиксировано шесть записей. Записи 1 и 2 (тип `I`) соответствуют вставке в таблицы *ports* и *clients*: заполнено только поле `new_data`. Записи 3 и 4 (тип `U`) соответствуют изменению: в них сохранены и старые, и новые значения (например, адрес электронной почты клиента изменён с `old@test.ru` на `new@test.ru`, число причалов порта — с 5 на 8). Записи 5 и 6 (тип `D`) соответствуют удалению: заполнено только поле `old_data`. Во всех записях указаны время операции и роль (`postgres`), от лица которой выполнен запрос.

2 ЗАДАЧИ ПО ШИФРОВАНИЮ ДАННЫХ

2.1 Таблица с секретными данными

Создадим таблицу `secret_data`, отдельную от основных сущностей. В ней хранятся токены доступа пользователей информационной системы. Оба содержательных столбца имеют тип `bytea`, так как результатом шифрования является двоичная строка.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS public.secret_data (
    id           integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    username     bytea, -- шифртекст (pgp_sym_encrypt возвращает bytea)
    secret_token bytea
);
```

2.2 Формирование ключа и шифрование содержимого

Для шифрования используется симметричный алгоритм AES-256 (функция `pgp_sym_encrypt` расширения *pgcrypto* с параметром `cipher-algo=aes256`). Ключ шифрования в информационной системе не хранится: он вычисляется как значение детерминированной хэш-функции SHA-256 от индивидуального пароля суперпользователя. Этот пароль не связан с паролем для входа в СУБД и известен только суперпользователю. Таким образом, без знания пароля восстановить ключ (и, следовательно, расшифровать данные) невозможно, поскольку хэш-функция является односторонней.

Пусть индивидуальным паролем суперпользователя будет `personal_admin_pass`. Вставим данные в зашифрованном виде; ключ вычисляется непосредственно в запросе и нигде не сохраняется:

```
INSERT INTO public.secret_data (username, secret_token)
SELECT pgp_sym_encrypt(v.u, k.key, 'cipher-algo=aes256'),  -- AES-256
       pgp_sym_encrypt(v.t, k.key, 'cipher-algo=aes256')
FROM (VALUES ('shipper_ivanov',    'token_4fsdf43fdaf43r2fdsa'),
             ('dispatcher_petrov', 'token_13gdfss4fa232fdsadfsa')) AS v(u, t),
     (SELECT encode(digest('personal_admin_pass', 'sha256'), 'hex') AS key) AS k;
```

Функция `pgp_sym_encrypt` при каждом вызове использует случайную соль, поэтому одинаковые открытые значения дают различающийся шифртекст.

2.3 Демонстрация невозможности чтения без пароля

Выведем содержимое таблицы. Даже пользователь с правом чтения таблицы (в том числе администратор) видит только шифртекст (рисунок 2).

```
SELECT * FROM public.secret_data;
```

	id [PK] integer	username text	secret_token text
1	1	shipper_ivanov	token_4fsdf43fdaf43r2fdsa
2	2	dispatcher_petr…	token_13gdfss4fa232fdsadf…
3	3	shipper_ivanov	token_4fsdf43fdaf43r2fdsa
4	4	dispatcher_petr…	token_13gdfss4fa232fdsadf…

Рисунок 2 — Содержимое таблицы «secret_data» в зашифрованном виде

Расшифровка выполняется только при указании ключа, полученного из верного индивидуального пароля:

```
SELECT id,
       pgp_sym_decrypt(username,     encode(digest('personal_admin_pass', 'sha256'), 'hex')) AS
    username,
       pgp_sym_decrypt(secret_token, encode(digest('personal_admin_pass', 'sha256'), 'hex')) AS
    secret_token
FROM public.secret_data;
```

При попытке расшифровать данные с неверным паролем СУБД возвращает ошибку (рисунок 3):

```
SELECT pgp_sym_decrypt(secret_token, encode(digest('wrong_pass', 'sha256'), 'hex'))
FROM public.secret_data;
```

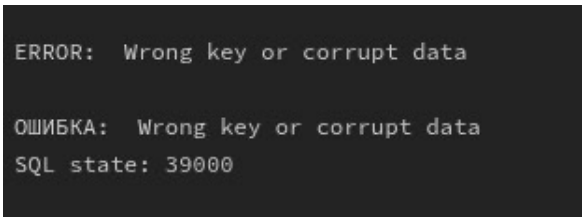


Рисунок 3 — Попытка расшифровки с неверным ключом

Следовательно, обладание правами администратора без знания индивидуального пароля не позволяет получить содержимое таблицы с секретными данными.

3 ЗАДАЧИ ПО РАЗГРАНИЧЕНИЮ ДОСТУПА В БД

3.1 Групповые роли для классов потребителей

Согласно системному анализу (ЛР1), в системе выделены два класса потребителей информации: «Клиент / Грузоотправитель» и «Диспетчер флота / Логист терминала». Для каждого из них создадим групповую роль без права входа. Дополнительно создадим групповую роль «Админ», которая должна иметь возможность просматривать все таблицы и представления базы данных.

```
CREATE ROLE client_group_role     NOLOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE INHERIT; -- Клиент /
    Грузоотправитель
CREATE ROLE dispatcher_group_role NOLOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE INHERIT; -- Диспетчер
    / Логист
CREATE ROLE admin_group_role      NOLOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE INHERIT; -- Админ
```

3.2 Назначение привилегий групповым ролям

Привилегии назначаются по принципу наименьших привилегий: каждой группе предоставляется доступ на чтение только к тем представлениям, которые необходимы ей для решения задач, определённых в ЛР1. Прямой доступ к базовым таблицам потребителям не предоставляется. Таблица логирования доступна только суперпользователю. Сводка привилегий приведена в таблице 1.

Таблица 1 — Привилегии групповых ролей

Групповая роль	**Доступные объекты (SELECT)**
client_group_role	представления *view_client_orders*, *view_client_tracking*
dispatcher_group_role	представления *view_voyage_manifest*, *view_container_safety_audit*
admin_group_role	все 9 таблиц основной схемы, таблица *secret_data*, все 4 представления
main_log	только суперпользователь

```
-- Клиент: только свои представления из ЛР1
GRANT SELECT ON TABLE public.view_client_orders   TO client_group_role;
GRANT SELECT ON TABLE public.view_client_tracking TO client_group_role;

-- Диспетчер: только свои представления из ЛР1
GRANT SELECT ON TABLE public.view_voyage_manifest        TO dispatcher_group_role;
GRANT SELECT ON TABLE public.view_container_safety_audit TO dispatcher_group_role;

-- Админ: все таблицы и представления, кроме лога лог( видит только суперпользователь)
GRANT SELECT ON TABLE
    public.clients, public.ports, public.ships, public.containers, public.orders,
    public.voyages, public.invoices, public.order_containers, public.voyage_orders,
    public.secret_data,
    public.view_client_orders, public.view_client_tracking,
    public.view_voyage_manifest, public.view_container_safety_audit
TO admin_group_role;
```

3.3 Индивидуальные роли пользователей

Создадим по одному пользователю для каждой групповой роли (с правом входа) и включим их в соответствующие группы:

```
CREATE ROLE shipper_ivanov    WITH LOGIN PASSWORD 'change_me_1' NOSUPERUSER NOCREATEDB NOCREATEROLE;
CREATE ROLE dispatcher_petrov WITH LOGIN PASSWORD 'change_me_2' NOSUPERUSER NOCREATEDB NOCREATEROLE;
CREATE ROLE admin_sidorov     WITH LOGIN PASSWORD 'change_me_3' NOSUPERUSER NOCREATEDB NOCREATEROLE;

GRANT client_group_role     TO shipper_ivanov;
GRANT dispatcher_group_role TO dispatcher_petrov;
GRANT admin_group_role      TO admin_sidorov;
```

3.4 Проверка работы системы разграничения доступа

Для проверки войдём под каждой из индивидуальных ролей с помощью команды `SET ROLE` и обратимся к отношениям базы данных. После проверки возврат к исходной роли выполняется командой `RESET ROLE`.

Роль «shipper_ivanov» (Клиент). Доступны только представления *view_client_orders* и *view_client_tracking*; обращение к остальным объектам завершается ошибкой `permission denied`.

```
SET ROLE shipper_ivanov;
SELECT * FROM public.view_client_orders;          -- OK
SELECT * FROM public.view_client_tracking;        -- OK
SELECT * FROM public.view_voyage_manifest;        -- permission denied
SELECT * FROM public.orders;                      -- permission denied
SELECT * FROM public.secret_data;                 -- permission denied
SELECT * FROM public.main_log;                    -- permission denied
RESET ROLE;
```

Роль «dispatcher_petrov» (Диспетчер). Доступны только представления *view_voyage_manifest* и *view_container_safety_audit*.

```
SET ROLE dispatcher_petrov;
SELECT * FROM public.view_voyage_manifest;        -- OK
SELECT * FROM public.view_container_safety_audit; -- OK
SELECT * FROM public.view_client_orders;          -- permission denied
SELECT * FROM public.clients;                     -- permission denied
SELECT * FROM public.secret_data;                 -- permission denied
RESET ROLE;
```

Роль «admin_sidorov» (Админ). Доступны все таблицы и представления, кроме таблицы логирования. Содержимое таблицы *secret_data* видно только в зашифрованном виде: расшифровка без индивидуального пароля завершается ошибкой.

```
SET ROLE admin_sidorov;
SELECT * FROM public.orders;                      -- OK
SELECT * FROM public.secret_data;                 -- OK, но только шифртекст
SELECT pgp_sym_decrypt(secret_token, encode(digest('guess', 'sha256'), 'hex'))
FROM public.secret_data;                          -- ошибка: Wrong key or corrupt data
SELECT * FROM public.main_log;                    -- permission denied
RESET ROLE;
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы для базы данных «Информационная система морских контейнерных перевозок» была разработана примитивная система защиты на уровне СУБД PostgreSQL.

В рамках работы были достигнуты следующие результаты:

1. Создана таблица-лог `main_log` и триггерная функция `logging()`. Для всех девяти основных таблиц созданы триггеры, фиксирующие вставку, изменение и удаление кортежей с указанием времени, роли и старых и новых значений. Работа системы продемонстрирована на нескольких таблицах и типах операций.
2. Создана таблица `secret_data` с секретными данными, содержимое которой зашифровано алгоритмом AES-256 средствами расширения *pgcrypto*. Ключ шифрования получается из индивидуального пароля суперпользователя с помощью хэш-функции SHA-256 и в информационной системе не хранится; без знания пароля прочитать данные невозможно даже при наличии прав администратора.
3. Реализовано разграничение доступа: созданы групповые роли для двух классов потребителей и администратора, а также индивидуальные роли пользователей. Привилегии назначены по принципу наименьших привилегий на представления, разработанные в ЛР1; таблица логирования доступна только суперпользователю.